



Data Leakage

Fuga de datos



Empecemos por el síntoma

Entrenás un modelo. En train da 99%. En test da 97%. Todo perfecto.

Lo ponen en producción y baja a 58%.

No hay ningún bug: el código corrió bien las dos veces.

Lo que pasó es que el modelo aprendió usando información que **no va a tener** cuando le toque predecir de verdad.



Empecemos por el síntoma

Lo que pasó es que el modelo aprendió usando información que **no va a tener** cuando le toque predecir de verdad.



Fuga de datos

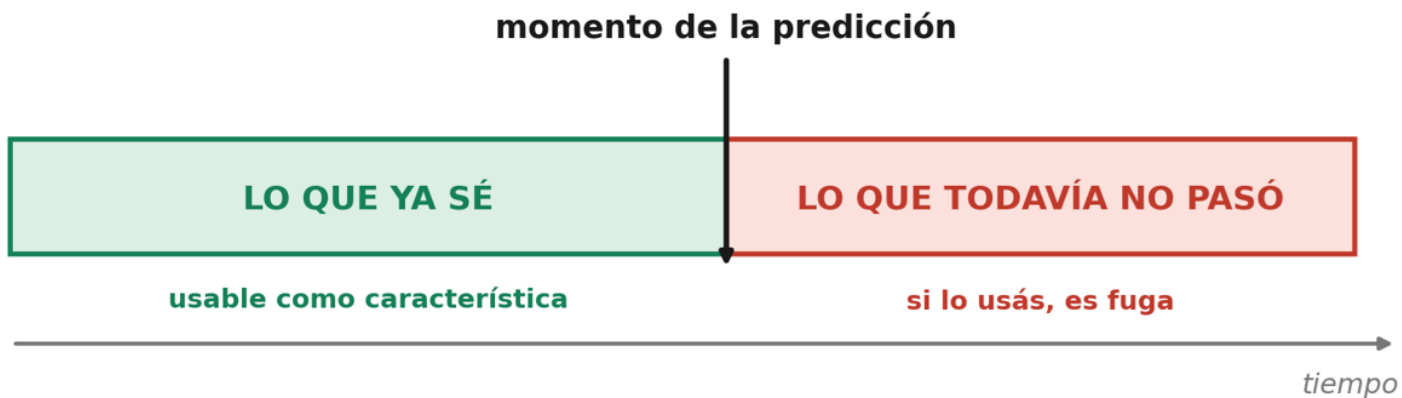
Los datos contienen información sobre lo que queremos predecir que **no estaría legítimamente disponible** en el momento real de la predicción.

El modelo la aprovecha, porque es lo que le pedimos. Y las métricas salen fantásticas.

Cuando el modelo sale a producción esa información no está, y el rendimiento se derrumba.

No es lo mismo que sobreajuste: ahí el test te avisa. Acá el test también está contaminado, así que no te avisa nada.

Fuga de datos



La pregunta de oro: ¿esta columna va a existir, con este valor, ANTES de que necesite predecir?



Se cuele de dos formas distintas

- 1. Información del futuro entre las características.** Una columna que en realidad es consecuencia de lo que querés predecir, o que se completa después.
- 2. Contaminación entre train y test.** El preprocesamiento mira datos de test antes de tiempo, y el test deja de ser una prueba honesta.

La primera se resuelve pensando el problema. La segunda, escribiendo bien el código.



Fuga de datos: ejemplos

El ejemplo más ingenuo: si la verdadera etiqueta de un conjunto de datos se incluyera como característica en el modelo.

1. Información del futuro



El caso clásico

Predice perfecto... porque el antibiótico
se receta DESPUÉS del diagnóstico



tiene_neumonía (lo que quiero predecir)	edad	peso	sexo	tomó_antibióticos
No	65	100	F	No
No	72	130	M	No
Sí	58	100	F	Sí
Sí	44	82	M	Sí

En producción, cuando el paciente todavía no fue diagnosticado, esta columna está vacía.



Ejemplo: consumo de energía eléctrica

Queremos predecir el consumo eléctrico, y la temperatura es un factor determinante.

Entrenamos con temperaturas **reales medidas**. Pero al pronosticar sólo tenemos **previsiones** de temperatura.

La variable existe en los dos momentos, pero no con la misma calidad: el error del pronóstico meteorológico se traslada al modelo.

Es una fuga parcial: el modelo confía en la temperatura más de lo que debería.



Ejemplo: ¿el usuario se va del sitio?

Un clasificador predice si el usuario se va a ir o se queda a leer otra página.

En el log aparecen columnas como **duración total de la sesión** o **total de páginas vistas**. Y funcionan buenísimo.

Claro: esos totales se calculan cuando la sesión ya terminó. En vivo es imposible conocerlos.

La corrección: reemplazarlas por páginas vistas **hasta este momento**, que sí se conoce mientras el usuario navega.



Cómo darse cuenta

La métrica es demasiado buena. Si tu modelo da 99% en un problema que nadie resolvió bien.

Una sola variable explica casi todo. Mirá la importancia de las características: si una domina, revisá cómo se genera.

Preguntá cuándo se completa cada columna. No basta con saber qué significa: hay que saber en qué momento del proceso aparece el dato.

2. Contaminación train-test



Fuga de datos: contaminación train-test

Un tipo diferente de filtración ocurre cuando no se tiene cuidado de distinguir los datos de entrenamiento de los datos de validación/prueba.

La validación debe ser una medida de cómo se desempeña el modelo en datos que no ha considerado previamente. Este proceso se podría dañar de maneras sutiles si los datos de validación afectan el comportamiento del preprocesamiento. A esto se le llama **contaminación de entrenamiento-prueba.**



Fuga de datos: contaminación train-test

Por ejemplo, se ejecuta el preprocesamiento (como ajustar un imputador para valores faltantes) antes de llamar a `train_test_split()`. El modelo puede obtener buenos puntajes de validación, pero puede desempeñarse mal en la implementación.

Se han incorporado datos de la validación o los datos de prueba en el entrenamiento. Si la ingeniería de características es más compleja, puede ser más riesgoso.



Fuga de datos: contaminación train-test

Si la validación se basa en una simple división entrenamiento-prueba, se debe excluir los datos de validación de cualquier tipo de ajuste, incluido el ajuste de pasos de preprocesamiento.



Dos casos que el split al azar rompe

Series de tiempo. Dividir al azar deja el test intercalado con el train: el modelo aprende de puntos posteriores para predecir puntos anteriores. Hay que cortar por fecha.

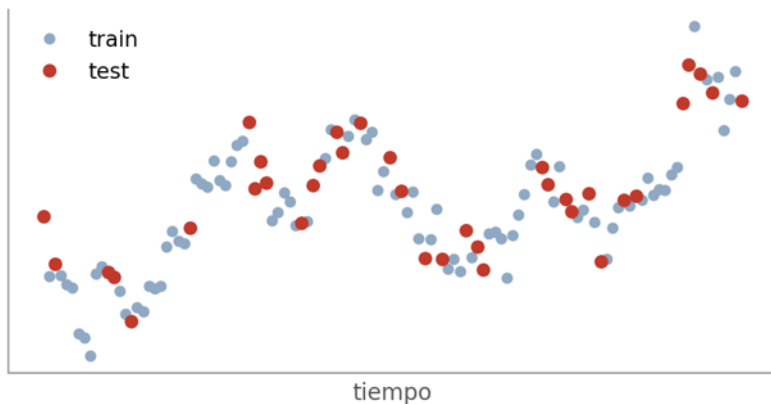
Filas agrupadas. Si el mismo paciente, cliente o usuario aparece en varias filas y unas caen en train y otras en test, el modelo lo reconoce en vez de generalizar.

En los dos casos el split al azar da métricas infladas por el mismo motivo de siempre: el test dejó de ser información nueva.

Series de tiempo

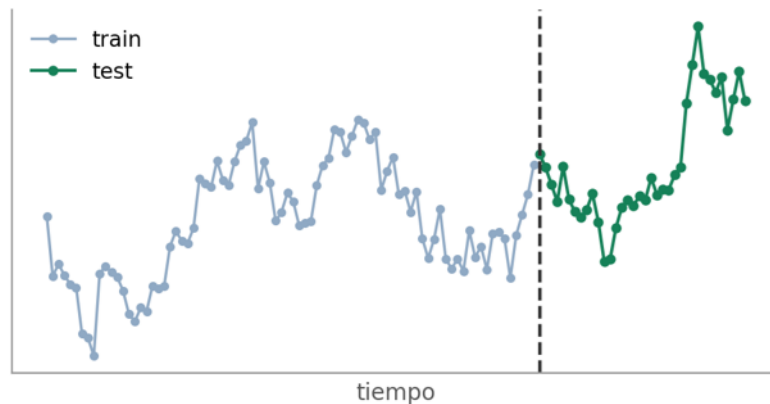
Con series de tiempo, dividir al azar es fuga

División al azar



El test queda intercalado: para predecir un punto el modelo ya vio el de al lado. Fuga.

División temporal



El test es todo posterior al train.
Es lo que va a pasar en producción.



Prevención de fuga

- Correcta división de train - test. Los pipelines de scikit-learn pueden ayudar.
- Analizar si las características estarán disponibles en tiempo real. Si no lo están, removerlas.
- En caso de que las variables sean leaky pero no se pueda desprender de ellas (caso temperatura en consumo de demanda) analizar el error de predicción con el error de pronóstico.
- Añadir ruido a las variables puede ser beneficioso.